Feeding coding agents docs from many repositories through a memory-mcp

meta-repository → document lifecycle → OpenSearch memory-mcp → knowledge graph

Jin · Ads Data Platform Team, Karrot (Daangn)

"Own your intelligence"

— Garry Tan. Source: Y Combinator Startup School 2026 keynote

The model is rented,
context is owned

The better the models get,
the more the differentiator
moves to context

Every model release is
a free upgrade to your library

You joined three months ago. Someone asks why the team made a call last year. What do you say?

I wasn't here back then?

There must have been a reason? lol

Is your team's documentation... Slack messages, by any chance?

Do you remember phone numbers?

I think history should work like phone numbers, too.

You know your wife's number by heart, of course haha

But your team lead's? You don't need to know it haha

"I know it starts with 010, at least haha"

One question

The dev docs your coding agents produce: how are you managing them?

01

We run a meta-repository

02

We keep them in each repository's docs

03

Docs are just leftovers

The philosophy that the source code is always the SSoT

What I dealt with every day

Every single session:
"This project's background is…, this term means…, last time we decided…"

The problem

The ads data platform team, in two sentences

Collect ad attribution and serve it

Build active-ad data and hand it to serving

Where the business logic concentrates. Side effects are hard to reason about

Schemas are coupled to the service layer → one repository can't show you the side effects

The problem

So this was my everyday situation

1 I have to read code in repositories I've never contributed a line to

2 I have to know which direction other teams' repositories are moving

3 And explain it all to the agent from scratch, every session

Skip the explanation and it invents history. Fixing that eats the day.

All I wanted was one thing

I wanted the AI to understand that repository's history on its own

- Don't make up history, even if I don't explain it every session
- When work on a repository starts, go read its history on your own

The approach

One stage at a time, as each problem showed up

Stage 1

meta-repository and directory structure

Trigger

Had to re-explain history every session

Result

INDEX.md only, no OpenSearch

Stage 2

Importing outside docs, and a lifecycle

Trigger

Docs lived outside too, and stale ones burned tokens

Result

60-day compaction, originals to S3

Stage 3

OpenSearch memory-mcp

Trigger

The meta-repository grew too big to explore

Result

Index hot only, reload cold from S3

Stage 4

knowledge graph DB

Trigger

Facts and valid rules had to stay current

Result

Knowledge in remote storage, not files

Stage 1 · meta-repository

meta-repository: one repository that holds the meta information of many

Meta information = four things

`spec`

`ADR`

`PRD`

`AI plan`

The plan the agent makes before it starts working

Stage 1 · meta-repository

The directory structure I proposed

```
meta-repo/
├── CLAUDE.md
├── repos/
└── docs/
    ├── repository/{repo}/
    │   └── INDEX.md
    ├── feature/{yyyy-mm-dd}_{topic}/
    │   └── INDEX.md
    ├── specs/{yyyy-mm-dd}_{spec}/
    │   └── INDEX.md
    └── knowledge/{topic}/
        └── INDEX.md
```

`repository/{repo}`
Its ADRs, background, term mappings. Assumed always current

`feature/{yyyy-mm-dd}_{topic}`
Features spanning repositories. Dated directory, episodic inside

`specs/{yyyy-mm-dd}_{spec}`
Human specs, AI specs, work plans. Dated directory, episodic inside

`knowledge/{topic}`
Facts and rules only, by topic. Not tied to a date

INDEX.md is the entry point for people, and the table of contents agents read first. **That's what let us run without OpenSearch.**
Once a search engine came in, maintaining INDEX.md was no longer needed.

Stage 1 · each directory has its own premise

Is this document tied to a date, or not?

repository/

Always current

Docs here are assumed to describe the present state

feature/ · specs/

Tied to a date

{yyyy-mm-dd}_ directories. Almost all episodic data

knowledge/

Not tied to a date

Topic directories. Facts and rules only. No lifecycle

Stage 1 · rules

Work in a new worktree; rules live in CLAUDE.md

Three reasons for worktrees

1. The agent can run several tasks at once; separate directories keep branches from mixing

2. Fixed working paths, so you can trace which change came from which task

3. codebase-memory-mcp detects worktrees and answers where code lives

`CLAUDE.md`

· Start by reading that repository's INDEX.md (from stage 3: search_memory first)

· When generating information from docs: "without inference"

· Which directory and frontmatter to save result docs with

· **The change history of these rules.** That's how docs made under old rules are told apart

Stage 1 · skill

One ticket in, and it runs all the way through

1. Jira · Linear ticket in
2. Do the work
3. Write the docs
4. Move the ticket status
5. Slack notification
6. Black-box test design → Linear comment
7. Deploy
8. e2e tests run automatically

I built the skill roughly. Sloppily. **I just asked the AI to make it.** What matters is #3: when the work is done, a doc is left in the meta-repository.

Stage 2 · importing docs from elsewhere

Docs lived outside the meta-repository, too

Building or changing a feature

Notion doc sync

Every doc in the managed repositories

→

`feature/`

Dated directories. Piles up as episodic

`knowledge/`

Nothing comes in directly. Only facts and rules that survive the lifecycle

Stage 2 · the document lifecycle

Compact docs past 60 days, move originals to S3

1

Target

Episodic docs (feature, specs) older than 60 days, by document age

2

Compaction

Write a new doc from them

3

Apply

New doc in feature or knowledge, or appended to one. With the S3 path

4

Move

Originals to S3. hot → cold

5

PR

The change is opened as a PR

Nothing is deleted. Originals go cold, what survives stays hot. At this stage hot/cold was still **files and S3 paths**, not OpenSearch. **Knowledge never goes through the lifecycle.**

Here compaction means rewriting docs to shrink them, not segment merging. Cold means archived in S3, not a data tier

Why stage 3: the meta-repository grew too big to explore quickly

Reading every similar doc is exactly the IR search engines have done for decades

If the agent does it

Walk the file tree, ag search → open → read → compare. It repeats for every similar doc; time and tokens pile up

If OpenSearch does it

One BM25 query. The agent gets only the top chunks

Stage 3 · requirements

Four conditions for the OpenSearch memory-mcp

The goal is querying episodic docs. Managing conflicts between facts and rules was out from the start.

#	Requirement	Why
1	Put doc files from GitHub or local disk into OpenSearch	The docs already exist as files in the meta-repository
2	Keep docs separated by team and by group	Even inside the ads group, each team has its own context
3	Have hot / cold areas	We may need context from a year or two ago. I wanted old history to stick around too
4	Store cold in S3, and load it straight into OpenSearch when needed	Too valuable to delete, too costly to keep online

Stage 3 · implementation

Six features we decided to build

1 OpenSearch runs on my **local machine** first. A shared team cluster is a later problem

2 **Put everything in, for now.** However badly built, it'll still beat ag search

3 **Watch the files.** Any change under docs gets re-indexed

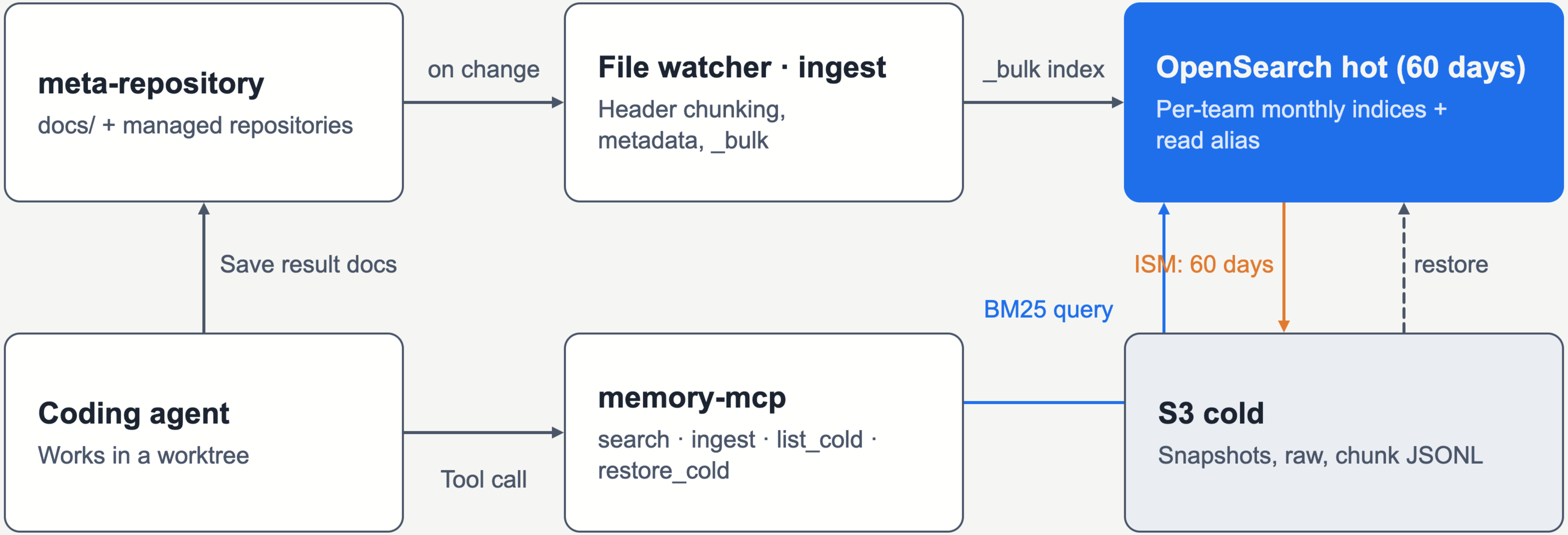
4 **BM25 only** at first, no embeddings. A cost thing. Embeddings come with remote mode

5 **Only hot** goes into OpenSearch. Same 60-day cutoff as stage 2

6 **When cold is needed,** download it from S3 and load it back into OpenSearch

Stage 3 · implementation

The agent asks OpenSearch, not the file tree



meta-repository
docs/ + managed repositories

on change

File watcher · ingest
Header chunking, metadata, _bulk

_bulk index

OpenSearch hot (60 days)
Per-team monthly indices + read alias

Save result docs

ISM: 60 days

restore

BM25 query

Coding agent
Works in a worktree

Tool call

memory-mcp
search · ingest · list_cold · restore_cold

S3 cold
Snapshots, raw, chunk JSONL

Stage 3 · worth a closer look

hot/cold borrows the log operations pattern as is

Episodic docs are append-only and pile up in time order. They behave just like log data.

Layer	Setup
Index	mem-{team}-{yyyy.MM} monthly indices; a read alias spans every hot one. Team split = index name + alias
Doc fields	Team, repository, doc type, time and path metadata + body. Time-range filters are key for episodic
Analyzer	nori (Korean morphological analysis) as the index's default analyzer
Chunking	By markdown header hierarchy. Never split inside a code block
ISM policy	hot 60 days → snapshot to S3 repository → delete index. Restore reloads an index. Not for knowledge docs
S3 layer	Raw originals, parsed chunk JSONL, snapshots. New mapping or embeddings: re-load JSONL via _bulk

Stage 3 · implementation

Four MCP tools were enough

`search_memory(team, query, time_range)`

BM25 search on the hot alias. Results are chunks with source_path and header_path

`ingest_path(path, team)`

Manual ingest for anything the watcher missed

`list_cold(team)`

Lists the cold indices sitting in S3

`restore_cold(index)`

Snapshot restore. For a query in the cold range, the agent first asks "restore it?"

Stage 3 · demo

Let's give it a task with no background

1 Save one ADR with the "active ad" term mapping under the meta-repository's docs. Watch the watcher index it in the log

2 Open a fresh agent session and give it an "active ad" task. No background

3 Watch the agent call search_memory and work from the source_path of the chunks it gets back

4 Compare it side by side with the same task run without memory-mcp

Stage 3 · measurement

No numbers.
You can tell once you use it.
(Probably..?)

The time and tokens spent digging through the file tree
and opening the same docs again and again dropped noticeably.

Why we moved to stage 4

Episodic can ignore conflicts. Knowledge never can

	Episodic docs	Knowledge docs
What it is	Work records piling up over time. ADRs, plans, meeting decisions	Facts that survived filtering. Term definitions, system invariants
Storage, IR	Inverted-index DB. Today, OpenSearch BM25	Graph DB (neo4j). Remote storage, not files
Lifecycle	Required. 60 days hot, then cold	Permanent, but needs an update history
Conflicts	Don't matter. Different records on one topic coexist	Always a concern. Pick A or B, or create C
Flow	Facts that survive compaction move to knowledge	Comes in only from episodic

Stage 4 · knowledge graph DB (neo4j)

To keep facts and valid rules current at all times, we moved knowledge into a graph DB

How it comes in

Facts and rules that survive stage 2 compaction

What we found

This information worked better in **remote storage** than in files

Demo

https://github.com/drakejin/workspace

How others do it · context engineering

Elastic tackled it with a "context engine"

> **"A context engine understands your data ahead of the question"**
>
> — Anish Mathur. Source: Elastic Search Labs, Context engineering for agents, 2026-10-08

The three problems Elastic names · our status

Per-query discovery cost	**Stages 1 & 3**
Context drift	**Stage 2**
Siloed knowledge	**Not yet · local OpenSearch**

The same three metrics · without → with

Metric	Elastic context engine	Our memory-mcp
Tool calls	33 → 9	Probably similar..?
Input tokens	2,279,253 → 653,511 (71% less)	Probably similar..?
Latency	213 s → 64 s	Probably similar..?

Elastic: one question about customer data, same model.
Us: dev-doc work. We didn't measure it.
Different data domains, so compare direction, not absolute values

Mapping this talk onto the context layer's continuous loop

Gather, Build, Retrieve: there. Improve: empty

Gather → **Build** → **Retrieve** → **Improve** → Back to Gather. Usage records become a new source

Stage	What this talk covers	What's missing	Status
Gather Read and find sources	Managed repository docs, Notion sync, feature work. All land in feature/ via the file watcher	Slack threads and ticket comments aren't sources	Covered
Build Store knowledge units	INDEX.md, frontmatter, header_path. Compaction moves facts and rules to knowledge, with S3 paths	Only two unit kinds, facts and rules. No data pitfalls or query tips	Covered
Retrieve Fetch relevant units	search_memory: BM25 (nori) on the hot alias, top chunks only. Cold restored after asking	No semantic search. Tools hit OpenSearch only, not the knowledge graph	Partial
Improve Usage becomes source	Change history of CLAUDE.md rules. People review compaction PRs	No usage collected. Misses and skipped search_memory calls never flow back	Missing

"If you're designing the context layer for your agents, build it on search." — Anish Mathur. Source: Elastic Search Labs, Context engineering for agents, 2026-10-08

The missing link · Improve

Today: a one-way pipeline. Usage must flow back for a loop

1

Usage records as a source

Log search_memory queries, results and tool calls with UBI; review them at compaction

2

Measure search quality

Score BM25 (nori) on query sets and judgment lists; run an experiment per change

3

A cluster the team shares

Move local OpenSearch to a team cluster. What one agent learns, the team uses

4

Ask once

Episodic and knowledge in one query, with semantic search on top of BM25

The pieces already exist in OpenSearch: **User Behavior Insights (UBI)** and the **Search Relevance Workbench** (query sets, judgment lists, experiments). They're tools in the official OpenSearch MCP server, too

Wrapping up

Wrap-up

memory-mcp alone isn't enough. Three things that worked well with it

codebase-memory-mcp

Keeps code structure as a graph, so it answers "which function produces this field?" without walking files. Docs go to OpenSearch, code goes here

Code history in comments

A member variable with a strange history gets a comment saying why. The closest context there is: read the moment the code is read

Separating episodic from knowledge

episodic: work records piling up over time. Has a lifecycle; different records on one topic may coexist

knowledge: facts that survived filtering. Permanent; conflicts must be settled into one

Wrap-up

Five things I want to say

1 **Start with a meta-repository.** A directory structure and INDEX.md are enough to begin. Agents can't find scattered docs, and what they can't find, they make up

2 **Give your docs a lifecycle.** That keeps them from becoming leftovers. Keep processing, compressing and updating them. When docs get big, leave IR to OpenSearch

3 **Run codebase-memory-mcp locally on your codebases.** It saves file-exploration tokens and time every session

4 **Memory design decides how well an agent performs.** [Fill in two quotes as "Quote" — quoted from whom. Source: …]

5 **Run knowledge in a graph DB.** Facts and rules fit remote storage better than files

Run knowledge, and you'll face this

When a fact from episodic collides with an existing one, who decides whether it's A, B or C?

My current answer

On a conflict, **the latest wins.** If it looks false, a person reviews it

What I don't know yet

Who decides it "looks false," and how?

How would you handle it?

I'm not here to hand out answers. I'm here to talk it through

Thank you

Questions and stories are welcome

Jin · Ads Data Platform Team, Karrot (Daangn)

https://github.com/drakejin/workspace

Expected Q&A · references

Questions you might ask, and my answers

Q Why not the official OpenSearch MCP server?

A I wanted team and org concepts, I didn't know the memory MCP well, and I wanted to learn.

Q At that scale, isn't ripgrep enough?

A I started with ripgrep. Watching it hunt for which directory to read was painful. The search-and-reread loop burned more tokens, and my hunch was OpenSearch handles nuance better across docs that share the same words.

Q Why neo4j for knowledge, not OpenSearch?

A It felt better to give OpenSearch only the inverted-index role, and nothing beyond that.

Q What if compaction drops or invents information?

A We haven't verified it. That's the blind spot of this demo. What could work? Maybe the coding agent reports its satisfaction to memory-mcp, and we use the UBI plugin..?

Q What if records on the same topic collide?

A In episodic, coexisting is normal and the latest wins. Only when moving to knowledge does a person decide. Not automated yet.

Q What if the agent never calls search_memory?

A It's a start-of-work rule in CLAUDE.md. If it still gets skipped, we're experimenting with a session-start hook that searches automatically.

Video	**Garry Tan: Own Your Intelligence**	`youtube.com/watch?v=eRrc1pUY5oU`
Blog	**Context engineering for agents · Elastic Search Labs**	`elastic.co/search-labs/blog/context-engineering-for-agents`
Repo	**Official OpenSearch MCP server**	`github.com/opensearch-project/opensearch-mcp-server-py`